

Lecture 4:

Asymmetric Cryptography

Eleftheria Makri | Security

04/03/2026



Universiteit
Leiden

Bij ons leer je de wereld kennen

Asymmetric Cryptography

- Quick Recap of Symmetric Cryptography
- Asymmetric Cryptography (Why do we need it?)
- Symmetric or Asymmetric Cryptography? → The best of both worlds
- Public-Key Cryptography
 - Key Exchange
 - Digital Signatures
 - Public Key Encryption
- Public-Key Infrastructure (PKI)
- TLS Protocol (Transport Layer Security)
- *Some Bonus today...*

Quick Recap of Last Week...

Symmetric Cryptography

Cryptography



0101101000011011
10011010100010
1011010101001111
1101010101100010111
1010101010011110101
1101010111101000110
1101010100010011110
1010110101010011011

Cryptanalysis



Cryptology =

Cryptography



+

Cryptanalysis



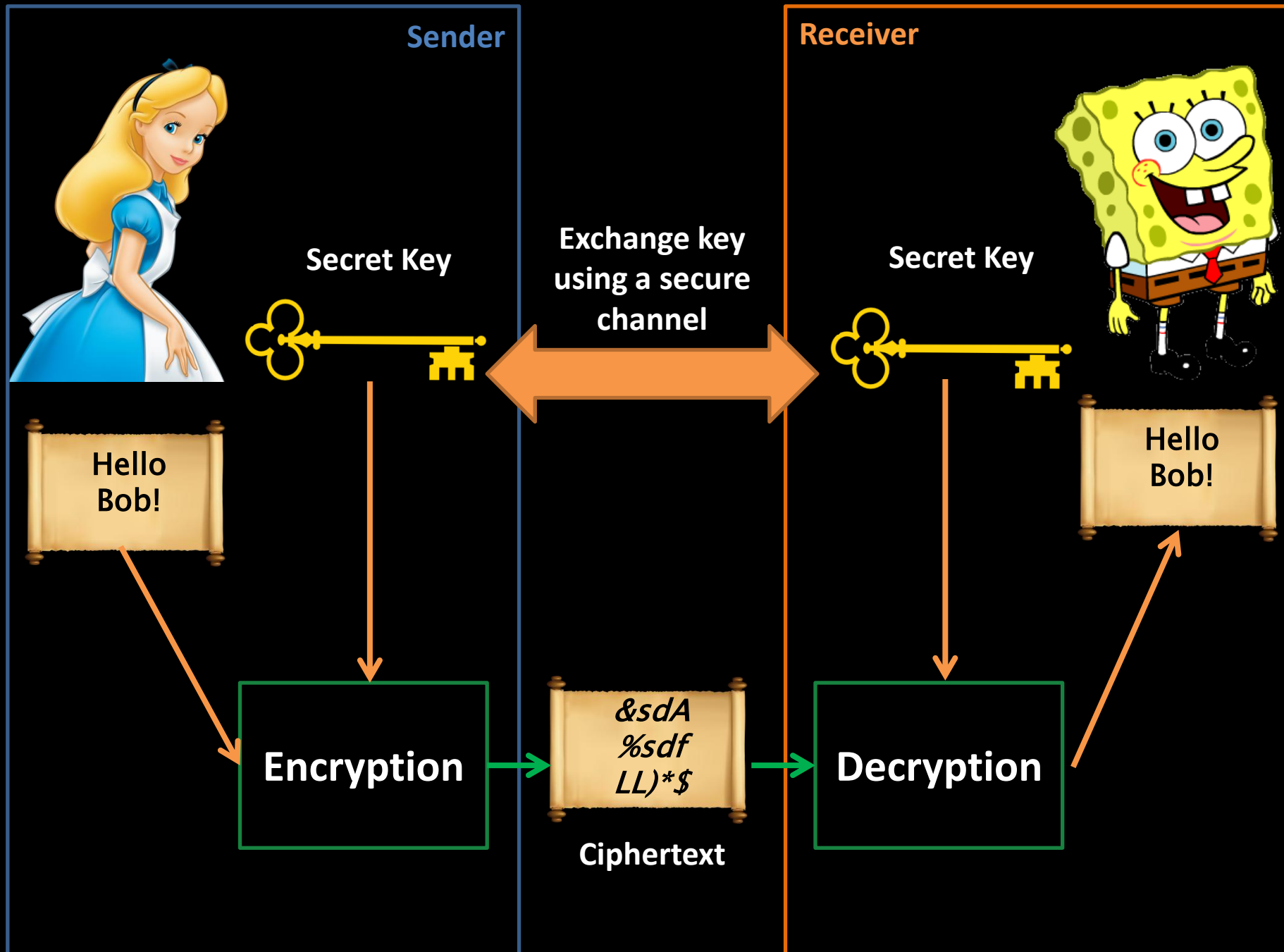
Important goals of cryptography

- Confidentiality = keeping information secret for everyone except for authorized entities
- Authentication
 - of entities = assuring the source of information
 - of data = assuring that the information has not been altered by unauthorized entities
- Non-repudiation = assuring that actions or commitments in the past cannot be denied

Classification of cryptography

- Symmetric-key cryptography:
 - Mainly for confidentiality
 - Sender and receiver have the same secret key
 - Block ciphers and stream ciphers
- Public-key cryptography:
 - Mainly for authentication (digital signatures) and key agreement
 - Sender and receiver have a different key (= public and private key)
- Hash functions:
 - Mainly for the authentication of data
 - With or without secret key

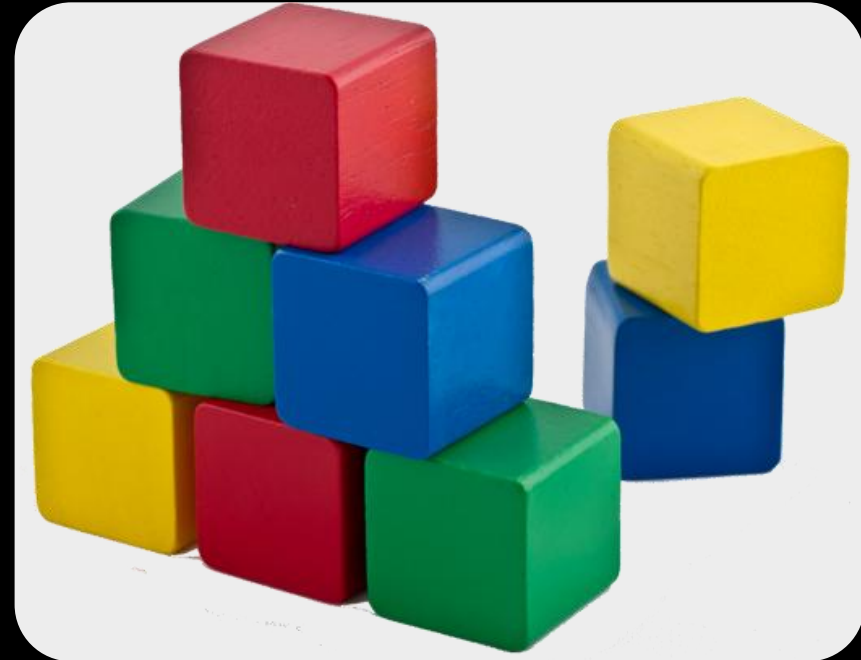
Symmetric-key cryptography



Stream Ciphers

and

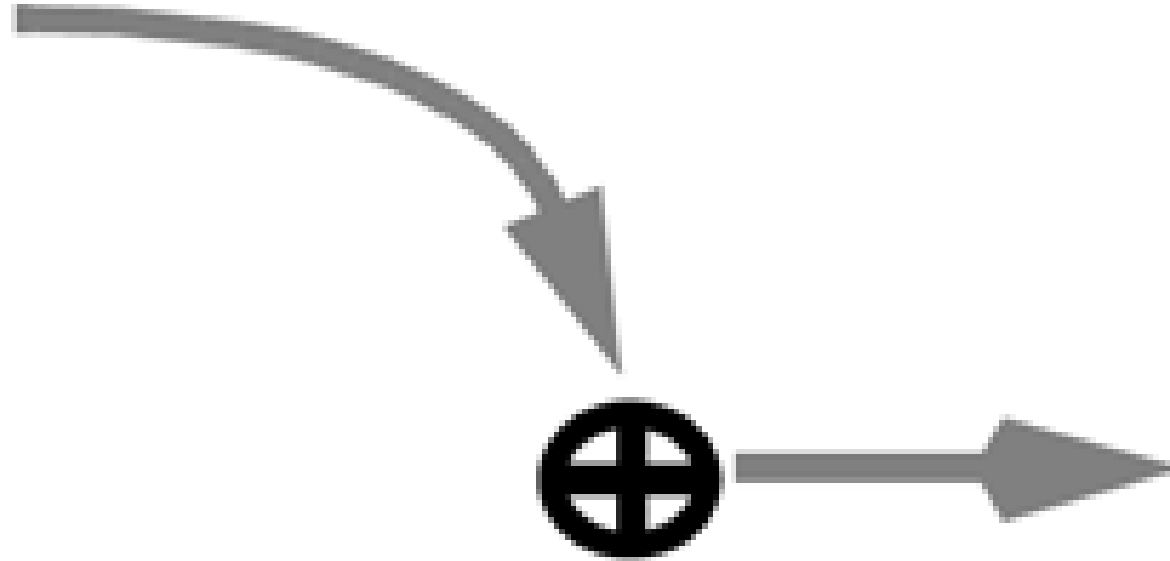
Block Ciphers



Stream Ciphers

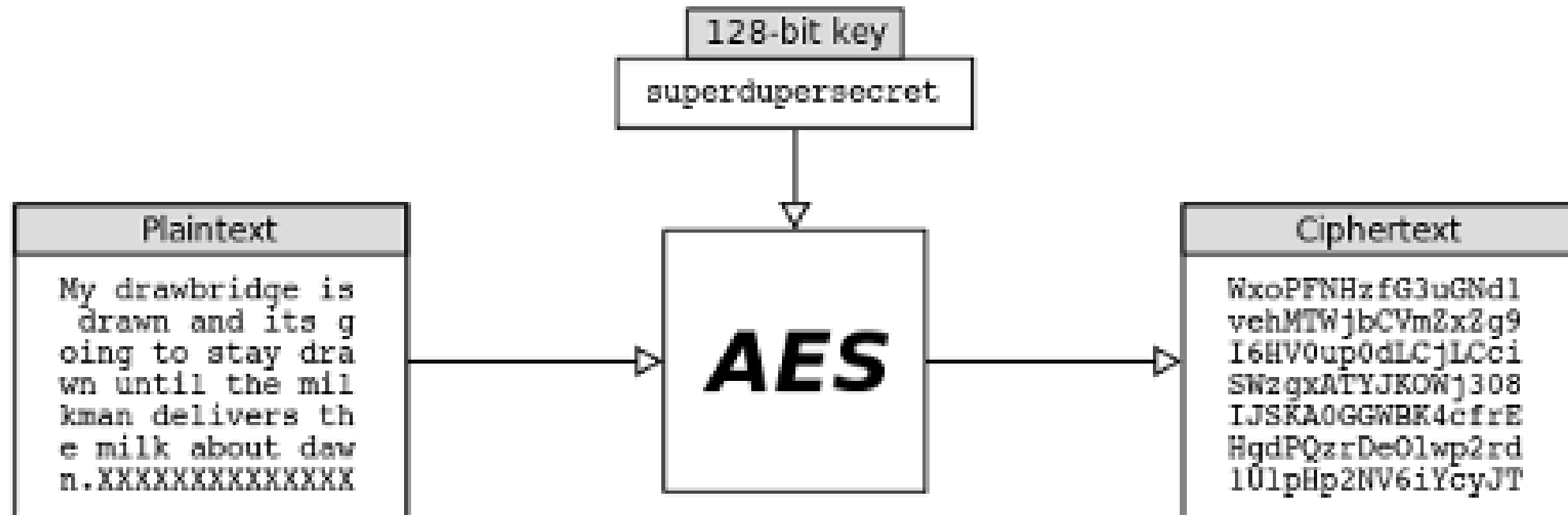
Key Stream
...10010101...

Plaintext
...01011001...



Ciphertext
...11001100...

Block Ciphers



Recommended key sizes for symmetric-key crypto

Protection	Symmetric	Factoring Modulus	Discrete Logarithm Key	Discrete Logarithm Group	Elliptic Curve	Hash
Legacy standard level <i>Should not be used in new systems</i>	80	1024	160	1024	160	160
Near term protection <i>Security for at least ten years (2018-2028)</i>	128	3072	256	3072	256	256
Long-term protection <i>Security for thirty to fifty years (2018-2068)</i>	256	15360	512	15360	512	512

Source: keylength.com (ECRYPT-CSA recommendations, 2018)

Hash Functions

Properties of hash functions

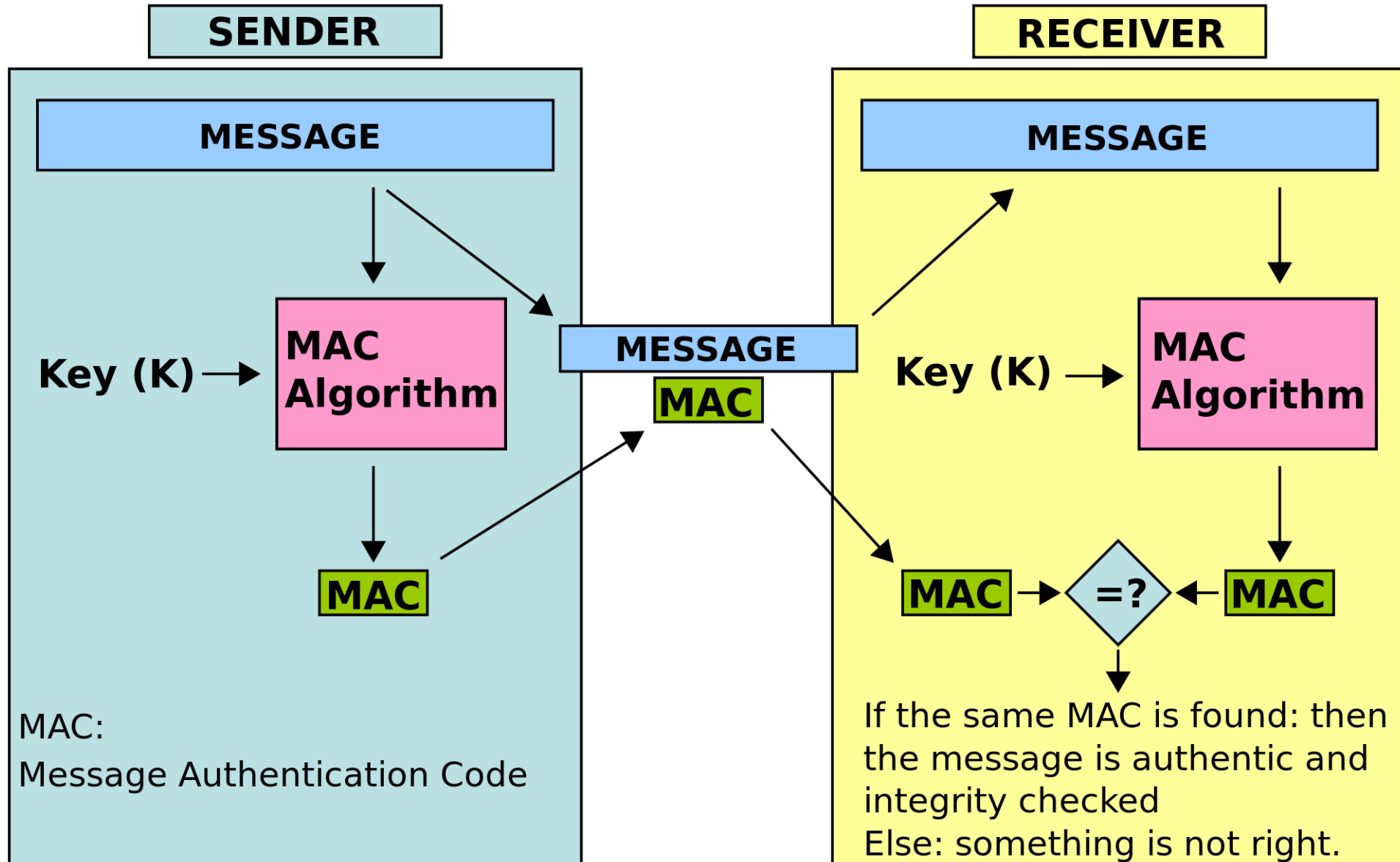
The ideal hash function for cryptographic applications has the following properties:

- It is **easy** to calculate the hash of a message
- Given the hash value h , it is **difficult** to find a message that has h as a hash value
= **pre-image resistance**
- Given the message m_1 , it is **difficult** to find a message m_2 that has the same hash value as the hash value of the message m_1
= **second pre-image resistance**
- It is **difficult** to find two different messages with the same hash value
= **collision resistance**

The use of a secret key

- Hash functions **without** a secret key are used to assure data integrity. They allow to detect changes in the transmitted messages.
- Hash functions can also be used for authentication (of data). In this case, a **secret key** is used, together with the message, and we call the function a Message Authentication Code (MAC).

Message Authentication Code – MAC

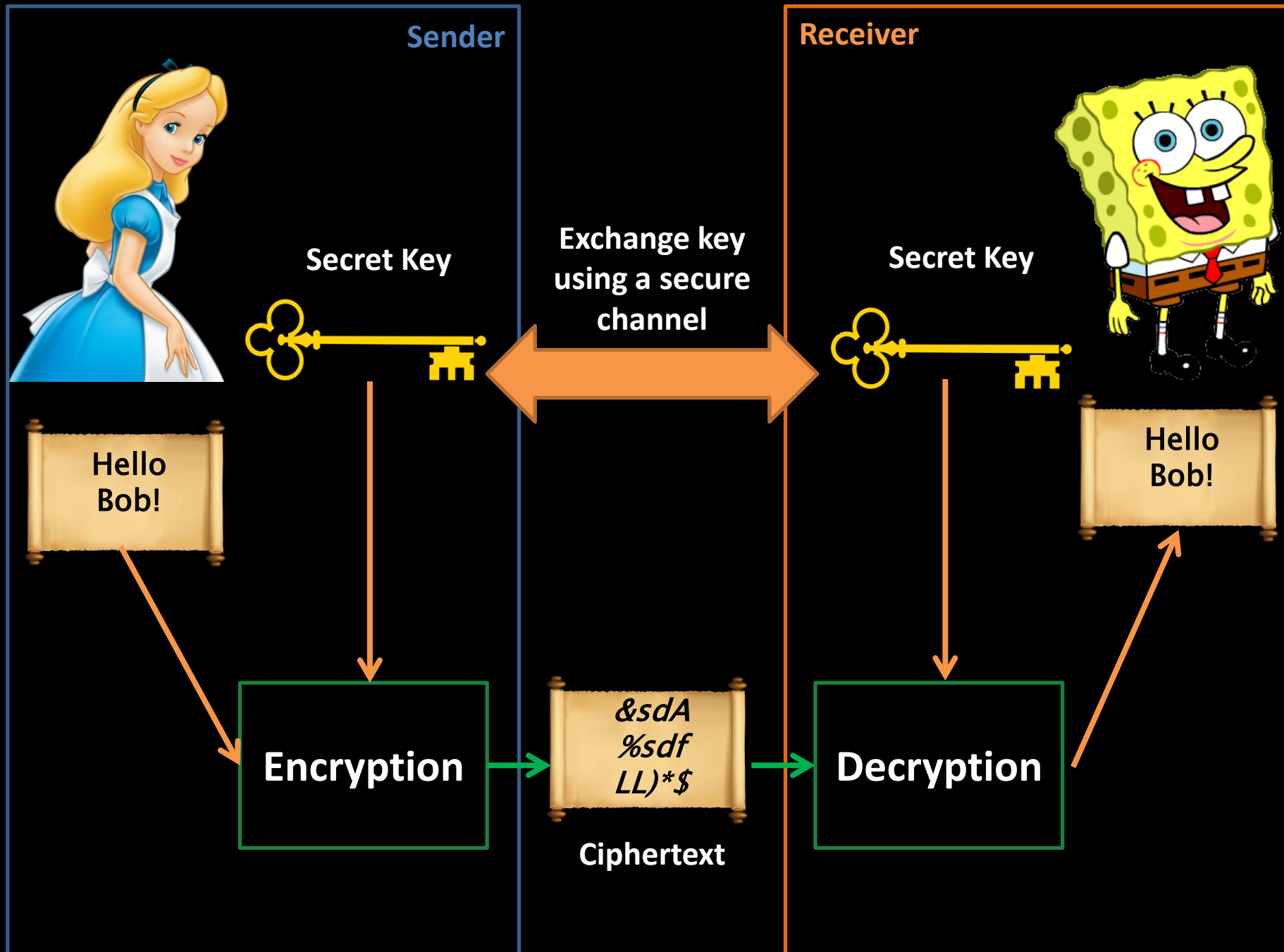


Recommended size of the hash value

Protection	Symmetric	Factoring Modulus	Discrete Logarithm Key	Discrete Logarithm Group	Elliptic Curve	Hash
Legacy standard level <i>Should not be used in new systems</i>	80	1024	160	1024	160	160
Near term protection <i>Security for at least ten years (2018-2028)</i>	128	3072	256	3072	256	256
Long-term protection <i>Security for thirty to fifty years (2018-2068)</i>	256	15360	512	15360	512	512

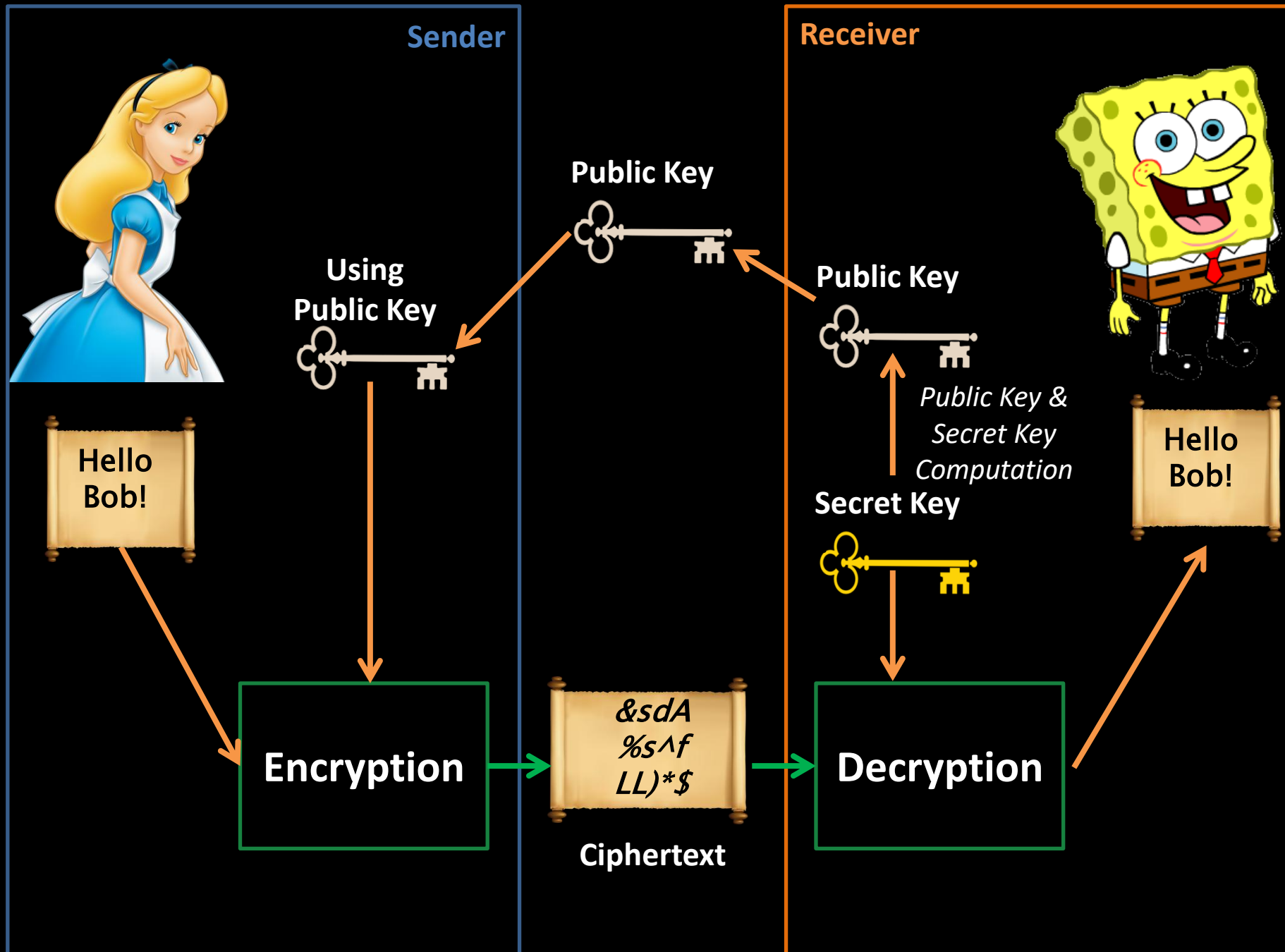
Source: keylength.com (ECRYPT-CSA recommendations, 2018)

Public-key cryptography

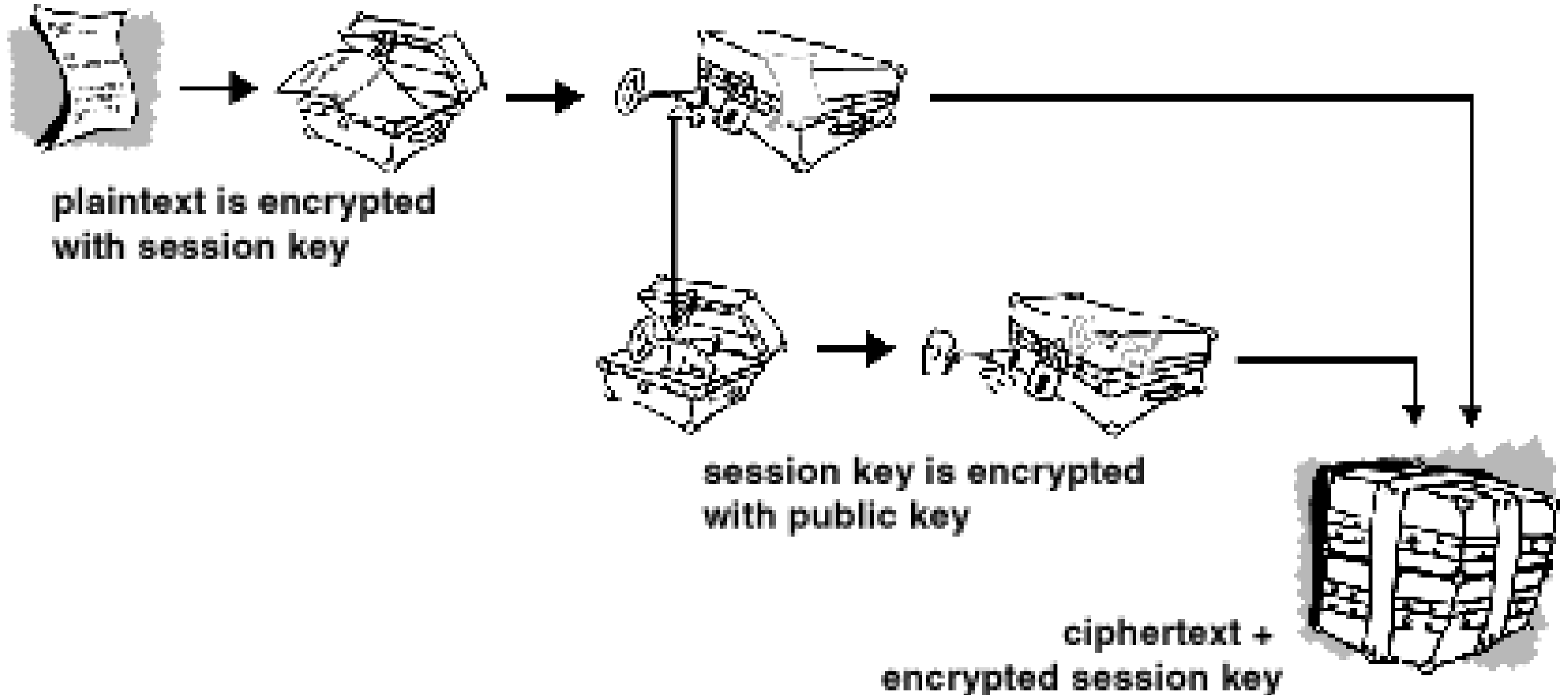




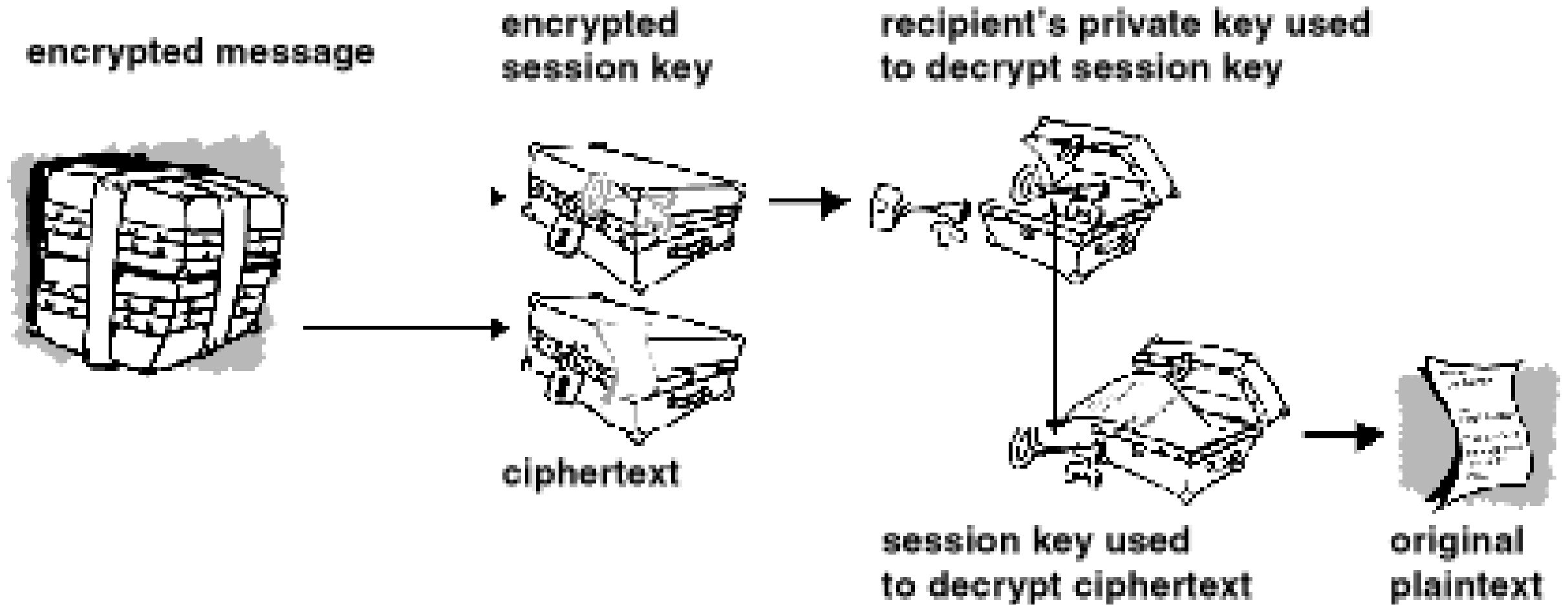




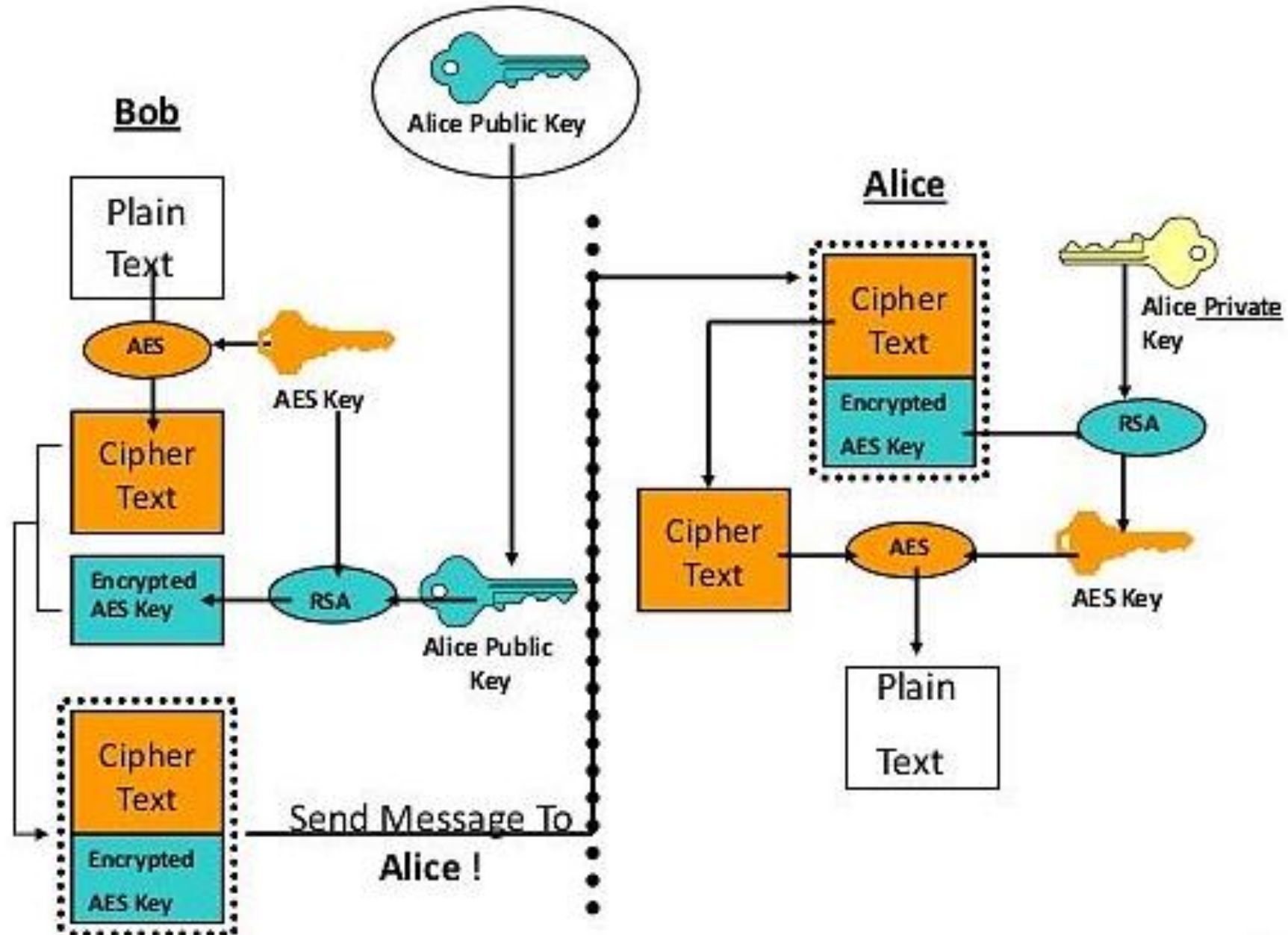
Hybrid Cryptography (Send)



Hybrid Cryptography (Receive)



Hybrid Cryptography (Combination)



Public-Key Cryptography

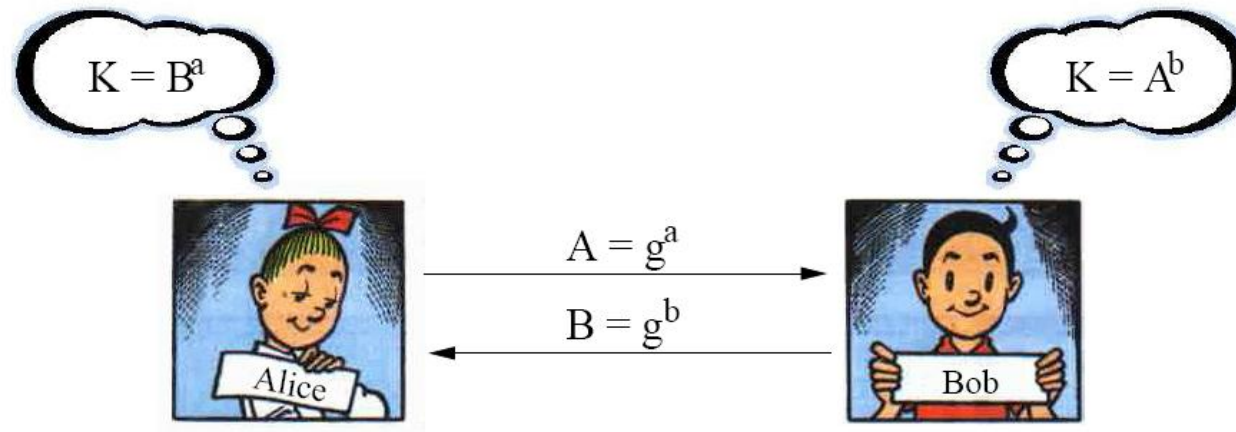
- Key exchange
- Digital signature
- Public-key encryption

Public-Key Cryptography

- **Key exchange**
- Digital signature
- Public-key encryption

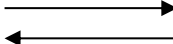
Diffie-Hellmann key exchange (1)

- Key agreement with Diffie-Hellman (DH)
 - E.g., to agree on the secret key that is needed to encrypt/decrypt messages with symmetric-key cryptography.
(all operations in the figure are modular operations)



Diffie-Hellmann key exchange (2)

All operations in the figure are modular.

Alice		Bob
$a \in [2, p-2]$		$b \in [2, p-2]$
$A = g^a \bmod p$		$B = g^b \bmod p$
		
$K = B^a = g^{ba} \bmod p$		$K = A^b = g^{ab} \bmod p$

It is very difficult to calculate the established key $K (= g^{ab} \bmod p)$ knowing only g , $A (= g^a \bmod p)$, $B (= g^b \bmod p)$ and the modulus p . This is called the Computational Diffie-Hellman problem (CDH). The CDH is related to the Discrete Logarithm Problem (DLP): it is very difficult to calculate a knowing only $A (= g^a \bmod p)$, g and the modulus p .

Recommended sizes for crypto based on DLP

Protection	Symmetric	Factoring Modulus	Discrete Key	Logarithm Group	Elliptic Curve	Hash
Legacy standard level <i>Should not be used in new systems</i>	80	1024	160	1024	160	160
Near term protection <i>Security for at least ten years (2018-2028)</i>	128	3072	256	3072	256	256
Long-term protection <i>Security for thirty to fifty years (2018-2068)</i>	256	15360	512	15360	512	512

Source: keylength.com (ECRYPT-CSA recommendations, 2018)

Elliptic curve cryptography

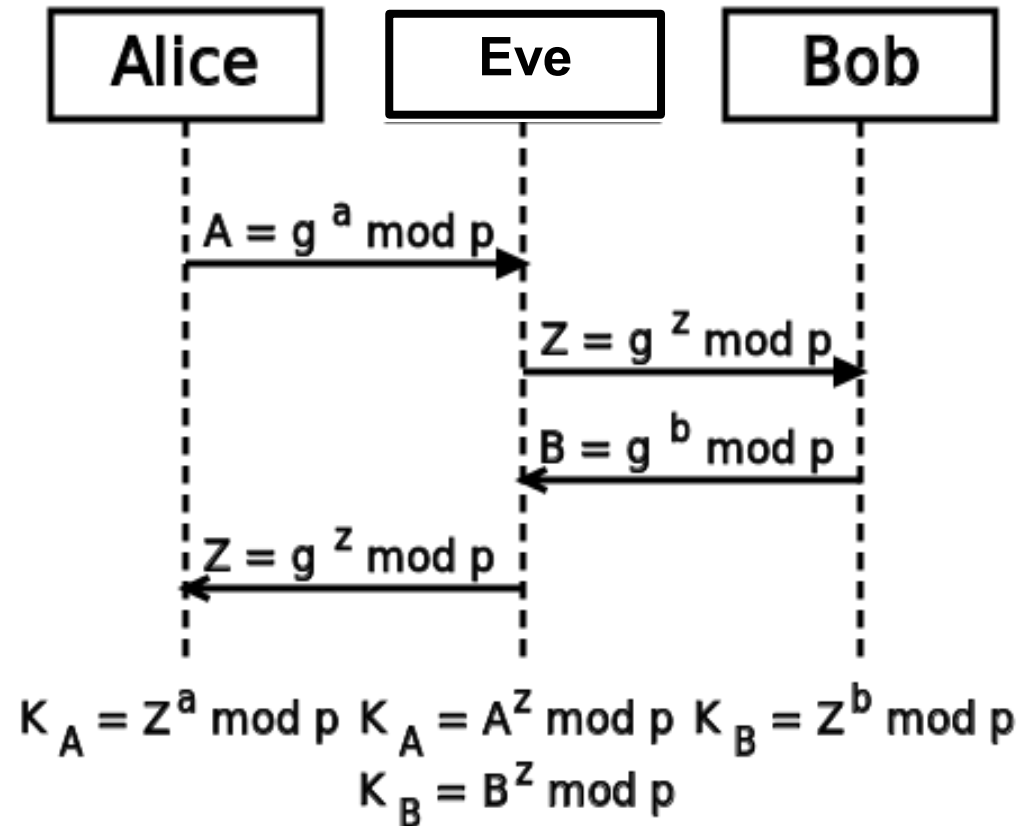
- In Elliptic curve cryptography (ECC) the modular exponentiation is replaced by a point multiplication
 - The points on the curve form a group with point addition as the group operation
 - The security is based on the Elliptic Curve Discrete Logarithm Problem (ECDLP)

Protection	Symmetric	Factoring Modulus	Discrete Logarithm Key	Discrete Logarithm Group	Elliptic Curve	Hash
Legacy standard level <i>Should not be used in new systems</i>	80	1024	160	1024	160	160
Near term protection <i>Security for at least ten years (2018-2028)</i>	128	3072	256	3072	256	256
Long-term protection <i>Security for thirty to fifty years (2018-2068)</i>	256	15360	512	15360	512	512

Source: keylength.com (ECRYPT-CSA recommendations, 2018)

Man-in-the-middle (MITM) attack on DH

- Eve pretends to be Bob in the communication with Alice
- Eve pretends to be Alice in the communication with Bob
- After the key agreement, Eve shares a key with Alice (K_A) and a key with Bob (K_B)
- In any further communication, Eve can decrypt all messages from Alice to Bob (or from Bob to Alice) with K_A (or K_B) in order to obtain the transmitted message.
- If Eve encrypts the message afterwards with K_B (or with K_A) and sends it to Bob (or Alice), Alice and Bob do not notice that Eve can intercept all messages.



Man-in-the-middle attack on DH: solution

- To prevent a MITM attack, Alice and Bob need to perform mutual authentication
- This can be done with a digital signature

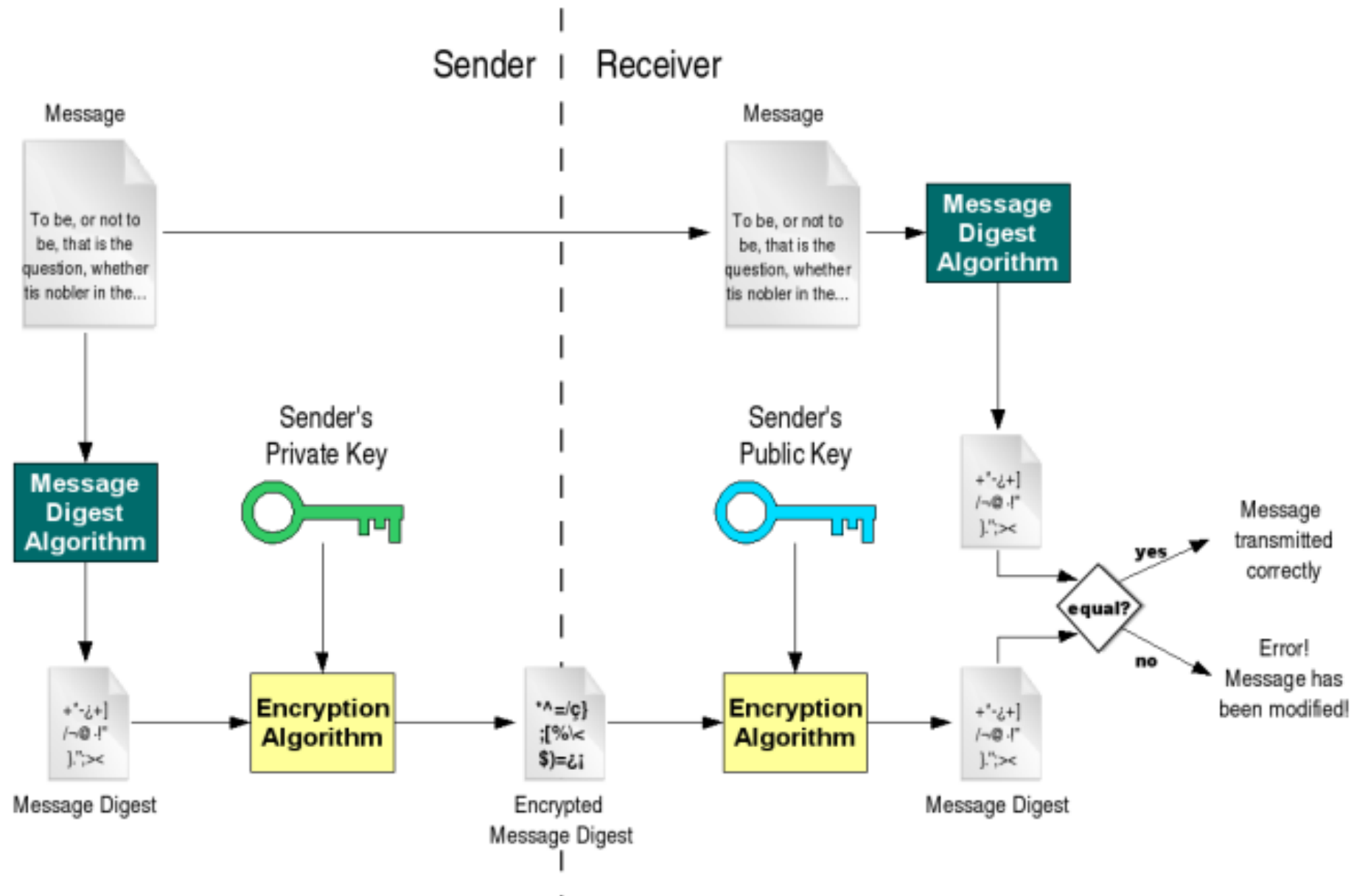
Public-Key Cryptography

- Key exchange
- **Digital signature**
- Public-key encryption

Digital signature (1)

- A digital signature assures the receiver of a message that the message has been sent by the claimed sender.
- To generate/verify a digital signature, a public/private key pair is needed.
 - The public key is known to everyone
 - The private key is kept secret by the owner

Digital signature (2)



Digital signature (3)

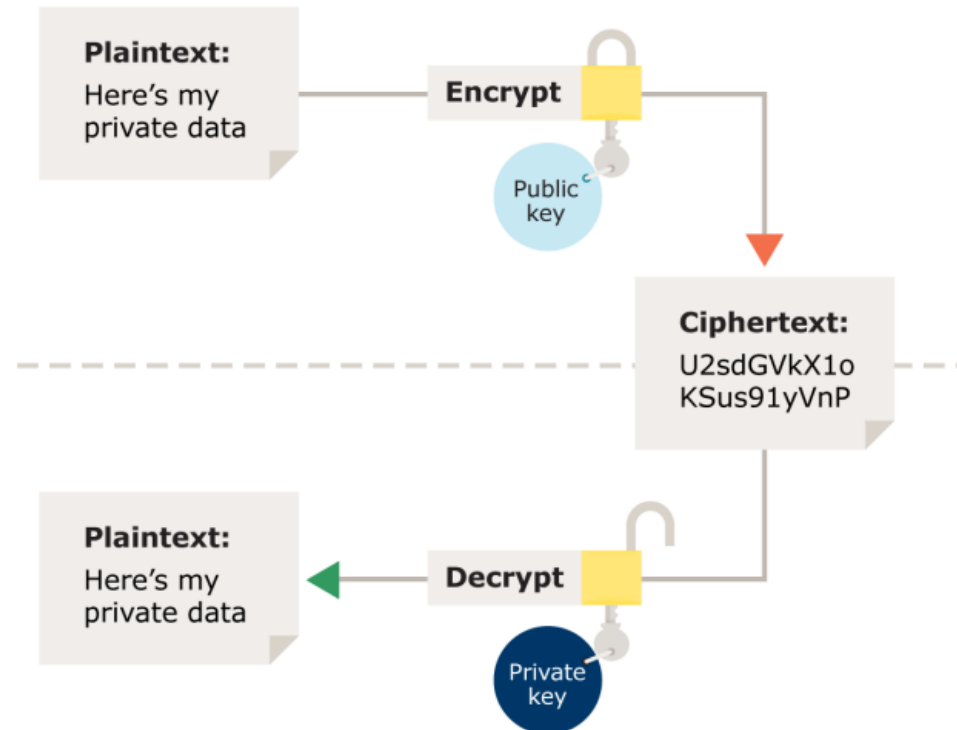
- Signature generation: The sender uses his private key to generate a signature. This is the encryption with the private key (of the hash value) of the message.
- Signature verification: The receiver uses the public key of the sender to verify the digital signature. The receiver does this by:
 - Decrypting the digital signature with the public key;
 - Comparing the result with (the hash of) the message.
- Algorithms for digital signatures:
 - DSA (Digital Signature Algorithm)
 - RSA (Rivest, Shamir, Adleman)
 - ECDSA (Elliptic Curve Digital Signature Algorithm)

Public-Key Cryptography

- Key exchange
- Digital signature
- **Public-key encryption**

Public-key encryption/decryption (1)

- Public-key encryption: The sender uses the public key of the receiver to encrypt the message.
- Public-key decryption: The receiver uses his private key to decrypt the ciphertext.



Public-key encryption/decryption (2)

- Public-key encryption: The sender uses the public key of the receiver to encrypt the message.
- Public-key decryption: The receiver uses his private key to decrypt the ciphertext.
- Example ElGamal encryption/decryption (security based on DLP):
private key: a
public key: (g, A) with $A = g^a$

Encryption of the plaintext m :

$(x, y) = E(m)$, with $x = g^k$ and $y = m * A^k$ (with k a random number)

Decryption of the ciphertext (x, y) :

$D(x, y) = (x^a)^{-1} * y = g^{-a*k} * m * g^{a*k} = m$

- Other example: RSA (see also exercises later)



Public-key encryption/decryption (3)

- Advantages of public-key encryption over symmetric-key encryption:
 - Different senders can use the public key of the receiver.
 - Only the receiver can decrypt the message using the private key.
- Drawbacks compared to symmetric-key encryption:
 - Public-key algorithms require more computation time and are therefore slower than symmetric-key algorithms.

Digital signatures vs. public-key encryption

- Public-key encryption using public-key cryptography
 - encryption: use the **public** key of the **receiver**
 - decryption: use the **private** key of the **receiver**

everyone can send, only the receiver can read

- Digital signature using public-key cryptography
 - generation: use the **private** key of the **sender**
 - verification: use the **public** key of the **sender**

only the sender can sign, everyone can verify

Use different key pairs for encryption and digital signatures!!!

Post-quantum cryptography

- Shor's algorithm allows to calculate discrete logarithms or to do integer factorization in polynomial time (assuming that quantum computers will become available)
- New algorithms that do not rely on discrete logarithms or integer factorization are needed
- Post-quantum cryptography investigates how we can implement crypto after/if quantum computers become ubiquitous
- Ongoing NIST Post-Quantum Cryptography competition

A working combination

Bringing it all together

- **Symmetric-key** crypto needs the least computation time – used for fast transfer of data.
- To establish a symmetric key over a public channel, we use **public-key** crypto. To generate and verify digital signatures, we use public-key crypto in combination with a **hash** function.
- In order to both encrypt and authenticate data, **authenticated encryption** is used.

Public-Key Infrastructure (PKI)

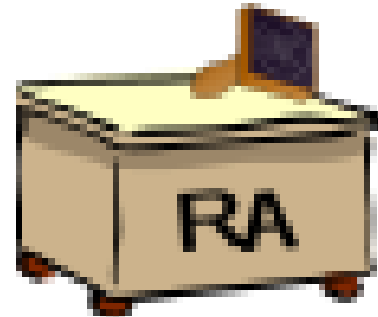
Certificates

- How can we be sure that a public key belongs to a certain entity?
- A certification authority (CA) gives a certificate to an entity that shows that the public key belongs to that entity.
- The CA is a trusted third party (TTP) that is trusted by the owner of the public key as well as other entities that use the public key.

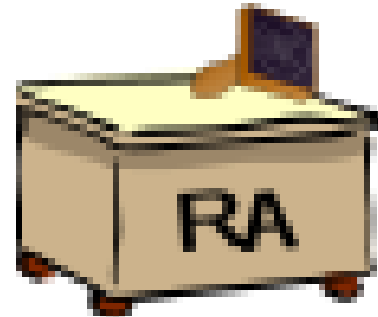
PKI



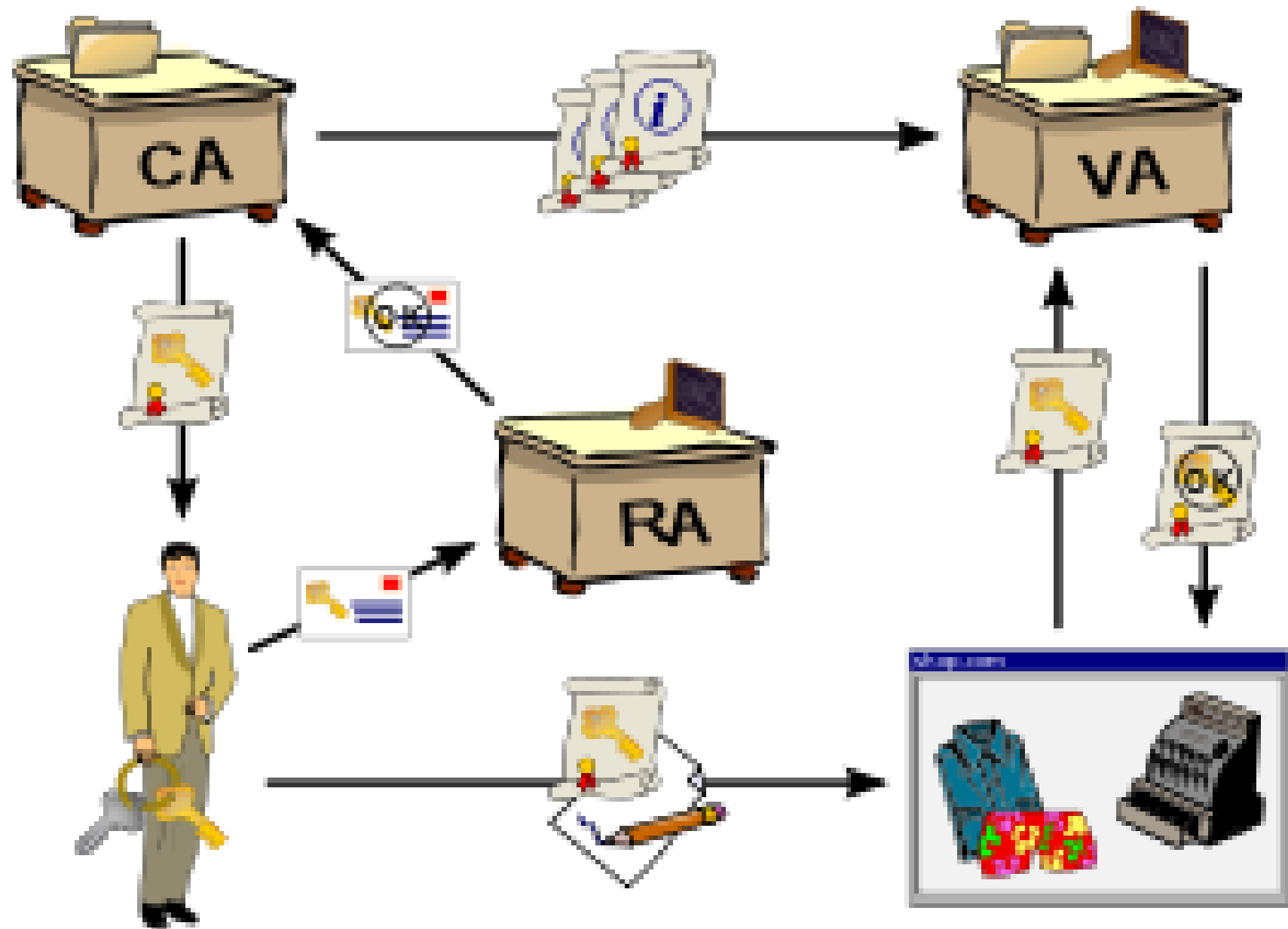
PKI



PKI



PKI



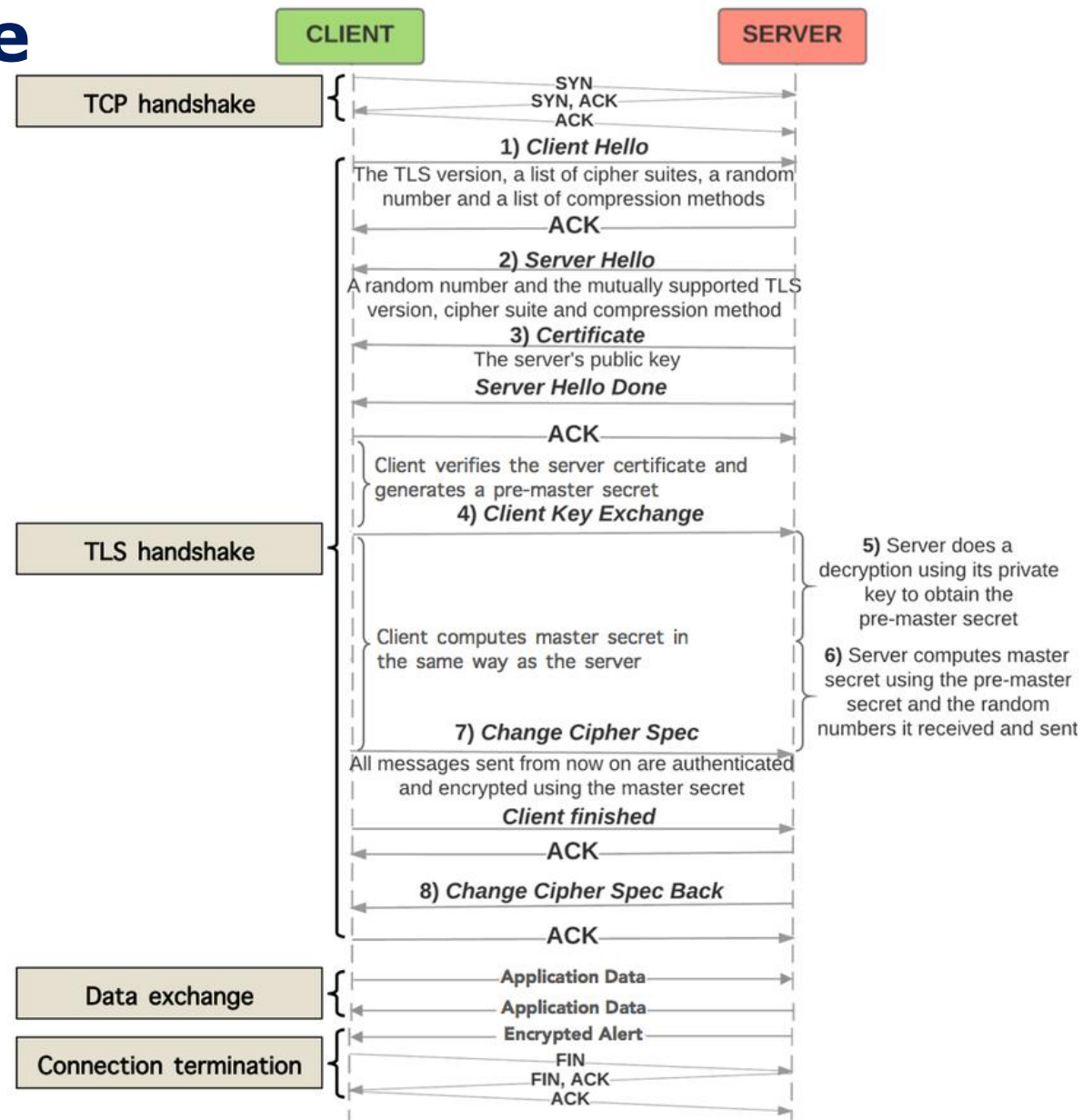
The TLS Protocol

SSL-TLS: Protocol for secure Web communications

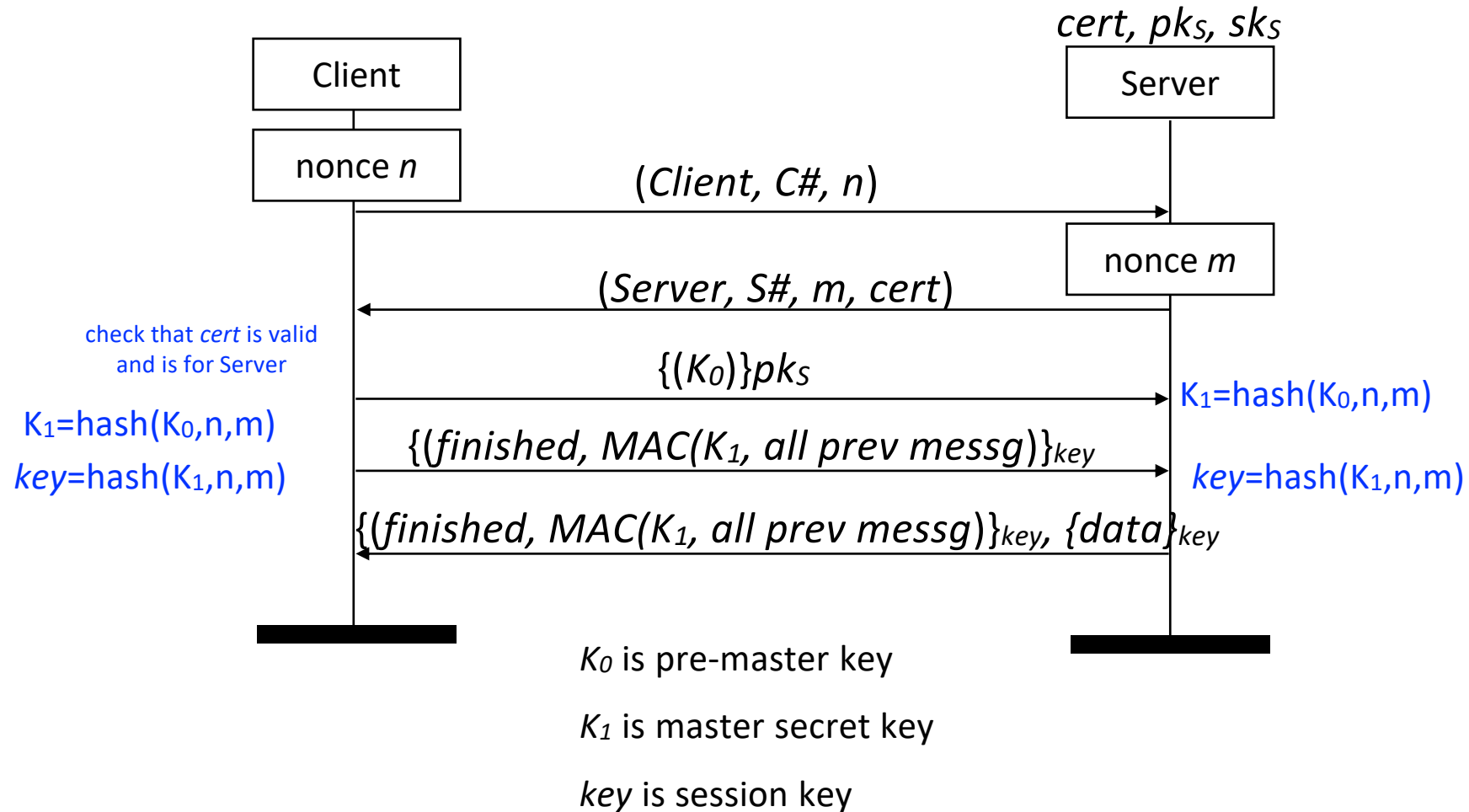
TLS stands for *Transport Layer Security* (new name)

- SSL stands for *Secure Socket Layer* (old name)
- HTTPS = *HTTP over TLS*
- TLS uses the PKI infrastructure and digital certificates of websites to ensure the following security goals:
 - secrecy of communication
 - authentication of parties
 - reliability (checks that messages are not corrupted in transit)
 - + good performance

TLS Scheme



TLS scheme: Session key establishment



MAC: message authentication code (like digital certificate but can be verified using shared key K_1)

RSA exercise

RSA

Key generation:

- Choose two different large primes (p and q) and compute $n = p \cdot q$
- Compute $\lambda(n)$
 - $= \text{lcm}(p-1, q-1)$
 - $= (p-1) \cdot (q-1) / \text{gcd}(p-1, q-1)$
 - (lcm = least common multiple)
 - (gcd = greatest common divider)
- Choose $e < \lambda(n)$ and relatively prime to $\lambda(n)$
- Calculate $d = e^{-1} \bmod \lambda(n)$

Public key = (e, n)

Private key = (p, q) or d

Encryption: $c = m^e \bmod n$

Decryption: $m = c^d \bmod n$

Example RSA

Given: $p = 19$, $q = 23$, $e = 13$, $m = 123$

Requested: d , c

Answer:

$$\lambda(n) = 18 \cdot 22 / \gcd(18, 22) = 18 \cdot 22 / 2 = 198$$

$$\begin{aligned} d &= 13^{-1} \bmod 198 \\ &= 61 \end{aligned}$$

(because $13 \cdot 61 \bmod 198 = 1$)

$$198 = 15 \cdot 13 + 3$$

$$13 = 4 \cdot 3 + 1$$

$$3 = 3 \cdot 1 + 0$$

$$1 = 13 - 4 \cdot 3$$

$$1 = 13 - 4 \cdot (198 - 15 \cdot 13)$$

$$1 = 61 \cdot 13 - 4 \cdot 198$$

$$1 = 61 \cdot 13 \bmod 198$$

$$d = e^{-1} \bmod 198$$

$$e \cdot d = 1 \bmod 198$$

Extended Euclidean algorithm

$$n = p \cdot q = 437$$

$$c = m^e \bmod n = 123^{13} \bmod 437 = 386 \text{ (check: } c^d \bmod 437 = 386^{61} \bmod 437 = 123 = m \text{)}$$

Homomorphism in RSA

Given: $c_1 = m_1^e \bmod n$ and $c_2 = m_2^e \bmod n$

Unpadded RSA allows us to compute:

$$c_1 * c_2 = (m_1^e \bmod n) * (m_2^e \bmod n) = (m_1 * m_2)^e \bmod n ,$$

which then decrypts to the product of the two plaintexts:

$$(c_1 * c_2)^d \bmod n = m_1 * m_2$$

- Malleability... allows for attacks
- But, also.... for Secure Computation schemes

The three states of data

Data in Transit



Data at Rest



Data in Use



The three states of data

Traditional Cryptography

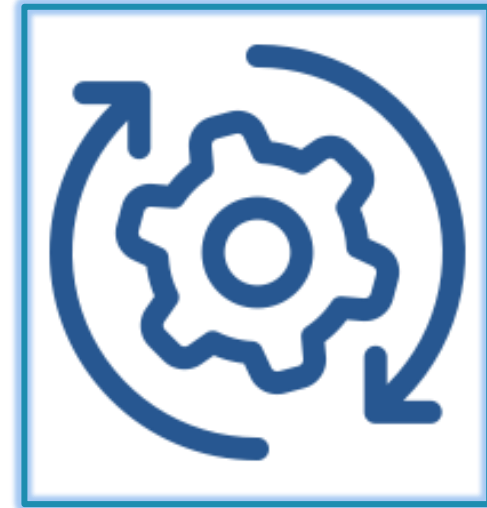
Data in Transit



Data at Rest



Data in Use



The three states of data

Traditional Cryptography

Data in Transit

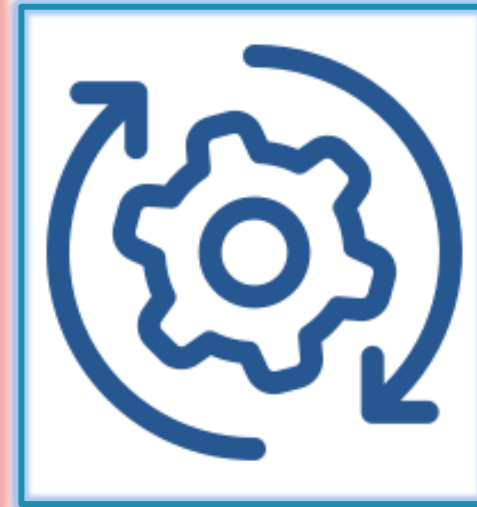


Data at Rest

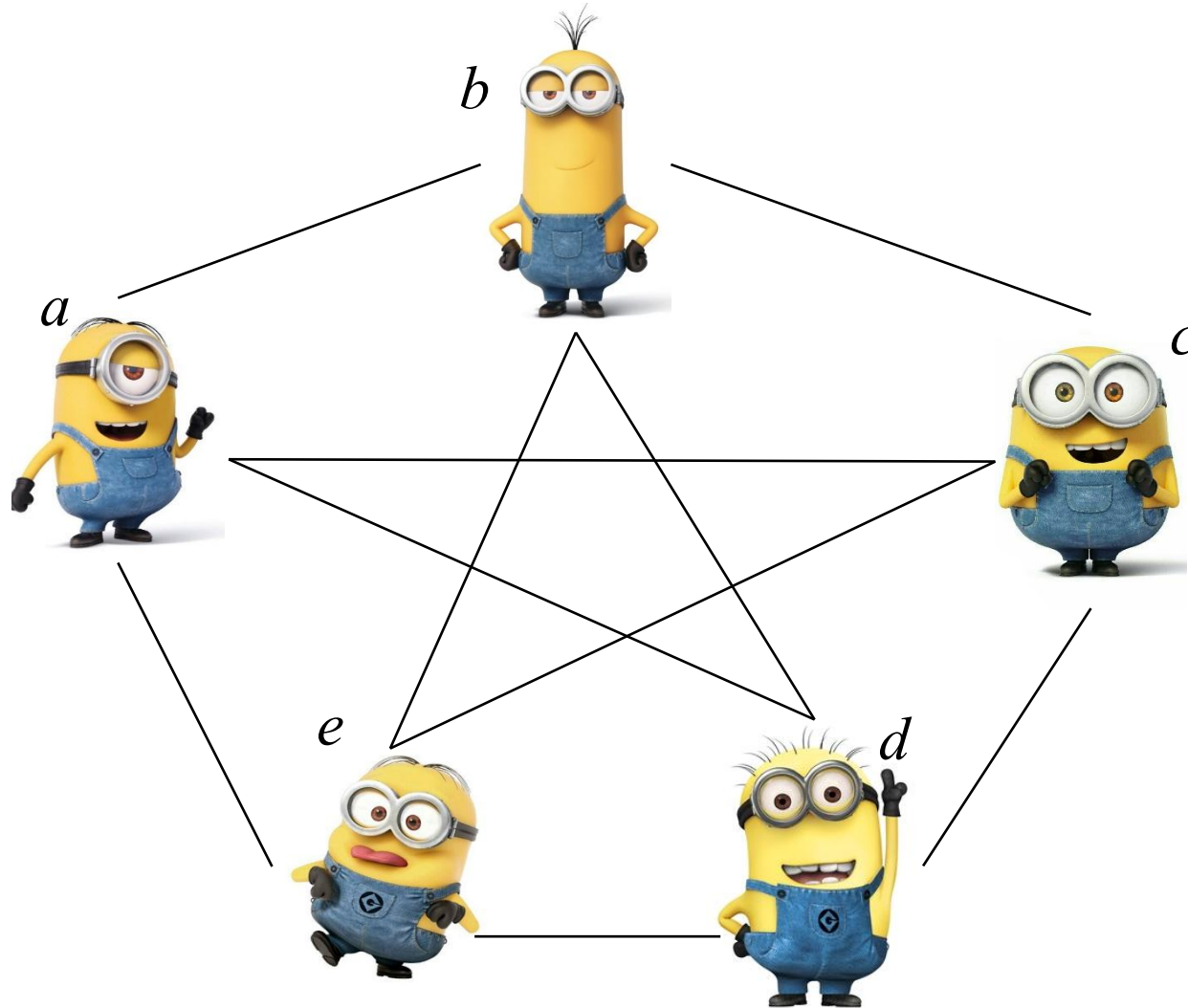


Secure Computation

Data in Use



Multiparty Computation – MPC



→ Securely compute $f(a, b, c, d, e)$.

Exercise RSA

Given: $p = 19$, $q = 17$, $e = 7$, $m = 321$

Requested: d , c

Solution:

$$d = 103$$

$$c = 195$$

Solution Exercise

Given: $p = 19$, $q = 17$, $e = 7$, $m = 321$

Requested: d , c

Answer:

$$\lambda(n) = 18 \cdot 16 / \gcd(18, 16) = 18 \cdot 16 / 2 = 144$$

$$d = 7^{-1} \bmod 144$$

$$= 103 \quad \leftarrow$$

(because

$$7 \cdot (-41) \bmod 144$$

$$7 \cdot (-41 + 144) \bmod 144$$

$$7 \cdot 103 \bmod 144 = 1)$$

$$144 = 20 \cdot 7 + 4$$

$$7 = 1 \cdot 4 + 3$$

$$4 = 1 \cdot 3 + 1$$

$$3 = 3 \cdot 1 + 0$$

$$1 = 4 - 1 \cdot 3$$

$$1 = 4 - 1 \cdot (7 - 1 \cdot 4)$$

$$1 = 2 \cdot 4 - 1 \cdot 7$$

$$1 = 2 \cdot (144 - 20 \cdot 7) - 1 \cdot 7$$

$$1 = 2 \cdot 144 - 41 \cdot 7$$

$$n = p \cdot q = 323$$

$$c = m^e \bmod n = 321^7 \bmod 323 = 195 \quad (\text{check: } c^d \bmod n = 195^{103} \bmod 323 = 321 = m)$$

References

- Nigel Smart, Cryptography Knowledge Area – The Cyber Security Body of Knowledge, Available at: https://www.cybok.org/media/downloads/Cryptography_v1.0.1.pdf (Chapters 2, 6, 7 and 8)
- Alfred J. Menezes, Paul C. van Oorschot and Scott A. Vanstone, Handbook of Applied Cryptography, Available at: <https://cacr.uwaterloo.ca/hac/> (Chapters 3, 4, 8 and 11 – the relevant parts therein)

Outlook

- We covered the basics of Symmetric and Asymmetric Encryption
- Next week we move to a new chapter: **Network Security**
- *However, we get back later in our course to discuss advanced cryptographic primitives that enable *Secure Computation Technologies***

Stay Tuned.....

* We discuss other topics first, interleaving the general cryptography topic, due to the upcoming practical assignments, which are related

Questions?



Universiteit
Leiden

Bij ons leer je de wereld kennen